KURUKSHETRA 2026 — THE STATEMENT YOU DOMINATE WITH

> **Written:** 16 Oct 2026, 14:20 IST · **Finalised:** 14:40 IST · **Kickoff:** 15:30 IST · **Freeze:** 17 Oct 15:30 IST
> **Status:** 🔒 **FINAL — do not re-open the track decision.** Read §13.2 (your speech), then §14 (next 55 minutes). **Wall clock:** 24h · **Your real build budget: 13–14 hours** (see §1) · **Team: proper-sized** (§12)
> **Companion:** `README.md` — event manual, rules, prize status, venue timeline

---

🏆 THE ANSWER: DI-01 — Debug Dojo

Track: Decentralized Intelligence · **Judging:** live demo + judge test pack · **Status:** 1 of 5 DI statements

> *"A coding coach that helps you find the bug without handing you the answer."* Runs your code safely → explains the failure in plain language → gives hints that get sharper without ever revealing the answer → learns which mistakes you keep repeating → hands you a new problem that forces you to fix it.

Why this one, in one sentence: It is the only statement on the board where the score is an **absolute checklist you can drive to ~100% yourself**, while every other team is fighting a relative leaderboard or a subjective panel — and it is the statement that **every other team's GPT tells them to skip.**

---

1. Why your 13–14 hours is the right number

I checked the official timeline and did the arithmetic. You're correct, and the plan must be built to *your* number, not to 24.

Window	Productive hours	Why it ends
15:30 → 18:30 (Day 1)	3.0	Eval 1 with mentors
18:30 → 23:00 (Day 1)	4.5	**Gate locks 23:00 — you're inside all night**
06:00 → 08:00 (Day 2)	2.0	Round 1 with the jury
08:00 → 10:00	2.0	Fix what judges flagged, then scope lock
10:00 → 14:00	4.0	Code freeze at 14:00
Total	**≈15.5h**	minus meals, transit, the three demos → **13–14h of real coding**

What this means strategically: the bottleneck is *not* writing code. With OpenCode and your subscriptions, generating a feature is nearly free. The scarce resource is **verification against the judge tests.**

So the winning move with your setup is not "build more" — it is **write the judge-test harness first, then drive it to green, repeatedly, for the whole 13 hours.** Every team that fails here fails because they started building features and tested at 14:00 on Day 2.

---

2. The competitive logic — why DI-06 is now dead

Signal	What it means
Your friend already took **DI-06**	The floor is filling
Every team's GPT gives the same recommendation	DI-06 is the most "web-dev approachable" DI statement — it has the largest crowd
DI-06 is judged on a **protocol checklist**	High score is achievable by *any* competent team, so scores compress together
DI-01 is the one people flinch at	Reading "sandbox red team, hidden-test secrecy, mistake taxonomy, 3 endpoints, 15 hostile programs" makes teams bounce
Net effect	A small field, absolute scoring, and a statement whose perceived difficulty is higher than its actual difficulty — **that is where you dominate.**

The rule: in a 24-hour event, difficulty you can beat is worth more than safety you share with 40 teams.

---

3. Why DI-01 specifically rewards *you*

Your advantage	How DI-01 converts it into points
13–14 focused hours	Spec is fully written for you — you never lose an hour deciding *what* to build
OpenCode + subscriptions	Code generation is free → you can afford the correct, boring, *complete* implementation instead of a clever half-one
You want depth, not breadth	The statement itself says: *"one programming language your team can support well."* It is the only statement that explicitly asks you to be narrow and perfect
Nothing built yet = no sunk cost	You can commit fully at 15:30
You want an emotional beat AND a crazy moment	See §13 — you get both, from the judges' own story

The dominance thesis in one line:

> Most teams will ship a chatbot with hints and call it a day. The score is awarded on **15 hostile programs, 12 outcome labels, secrecy checks, a leak test, a scripted learner, and 2 blind human raters.** If you build to the checklist, you finish in the 90th percentile *because most teams never read it twice.*

---

4. When to switch — the decision table

Your situation	Instead pick	Note
~~**Only 2 developers**~~	~~**W-PS-2**~~	**Retired — you have a proper team.** §12 scales: add a 5th owner as harness operator
You'd rather risk everything on **crazy** than win on **perfect**	**DI-04 Chimney Census** — see §16	Only if GPU + 4 GB download are confirmed within 60 min
Nobody can get an LLM API key working	**OP-05 Verified Course Material Studio** (Open)	Note: your code becomes TechSummitTrainers' property (§16)
You need a Gemini/Claude/OpenAI key and cannot get one	Stop and ask an organiser	The statement explicitly allows any model API — ask if sponsor keys exist
Strong DOM/UI, weak Python backend	W-PS-2	DI-01 is a Python-backend project with a UI on top

---

5. What DI-01 actually requires (exact numbers — memorise these)

5.1 Content

- **≥10 challenges**, across **≥3 difficulty levels**, covering **≥4 concepts**
- Each challenge: description · starter code · concept tags · **≥3 visible tests** · **≥5 hidden tests**, including edge cases
- Must include **these four judge challenges, exactly as specified** (translate signatures to Python):

Signature	Returns	Inputs
`last_k_total(marks, k)`	total of last k marks, or of all marks if k > list	`marks`: list of ints, possibly empty; `k`: int ≥ 0
`count_passes(marks)`	how many marks are ≥ 40	`marks`: list of ints, possibly empty
`first_n_squares(n)`	`[1, 4, 9, …, n²]`, or `[]` when n is 0	`n`: int ≥ 0
`middle_three(items)`	middle three items, or the whole list if < 3 items	`items`: odd-length list, or list with < 3 items

> So: 4 judge challenges + **6 more you write yourself**, minimum.

5.2 Execution

- Learner code **never runs unrestricted.** *"Full sandboxing is not required; a constrained local test runner is acceptable, **and stronger isolation scores higher**."*
- **Four outcome labels, exactly one per test:** `passed` | `wrong_output` | `runtime_error` | `timeout`
- A run killed by a **memory, output or disk limit** counts as `runtime_error` **and must say which limit**

5.3 Secrecy

- Hidden tests stay hidden: never reveal hidden **input, expected output, program output, or exception message**
- An explanation may name the *kind* of input — *"a case where k is larger than the list"* — **never the values**
- The UI must not receive hidden data either (judges inspect browser traffic)

5.4 Hints

- **≥3 hints per failure**, each more specific than the last
- **No hint may reveal the complete solution, even when the learner asks for it**
- **No hint contains code** — this is enforced by a test, see §7.4

5.5 Mistake taxonomy

Must include these **five, kept distinct** (you may add more):

`off_by_one` · `unhandled_short_input` · `print_not_return` · `boundary_comparison` · `mutates_input`

- A category is **recurring** once it appears in **≥2 different challenges**
- Show recurring mistakes to the learner, and choose/generate the next challenge to practise it, **saying why**

5.6 Mastery

- Mastery estimate **per concept**, updated from the learner's work
- **A pass without hints counts for more than a pass after hints**

5.7 The three API endpoints (judges reach these from the start of judging)

Endpoint	Body	Returns
`POST /api/submit`	`{learner_id, challenge_id, code}`	outcomes + explanation (JSON shape in §5.8)
`POST /api/hint`	`{submission_id}`	the **next** hint
`GET /api/learner/{learner_id}`	—	recurring mistakes, mastery per concept, next challenge **with the reason**

5.8 The exact JSON shape (from the statement — copy this)


```
{
  "submission_id": "s_0142",
  "learner_id": "rohan",
  "challenge_id": "last-k-total",
  "visible": [
    {"test":1,"call":"last_k_total([10, 20, 30, 40], 2)","expected":"70","got":"70","outcome":"passed"}
  ],
  "hidden": [
    {"test":1,"outcome":"passed"},
    {"test":3,"outcome":"runtime_error"},
    {"test":4,"outcome":"wrong_output"}
  ],
  "mistake_tags": ["unhandled_short_input"],
  "explanation": "All 3 visible tests pass, but 3 of the 6 hidden tests fail: one crashes and two give the wrong total. All three are ca
}
```


Note what is **absent** from `hidden[]`: no `call`, no `expected`, no `got`, no exception text. Only the outcome. That is the secrecy test in one object.

5.9 Rules & deliverables

- **One** programming language for learners → **Python.** Any stack for the platform
- Challenge content: own, or openly licensed with attribution. **Hidden tests must be your own**
- AI: any model for explanations/hints. **Log every prompt and response — judges may ask to see them**
- Learner data: **pseudonymous accounts only. No real student data**

Hand in by the freeze:

- [ ] Repo + README + **one command that starts everything**
- [ ] Live URL, **or a laptop the judges can use**, with the API endpoints reachable **from the start of judging**
- [ ] **Your mistake taxonomy and how each category is detected, on one page**
- [ ] **5-minute demo, then a viva**

---

6. The six judge tests → your test harness

Build this harness first, before any feature. It is your scoreboard for the next 13 hours.

#	What the judges do	What you must guarantee	Your self-test
T1	Submit **~15 hostile programs**: network, files, secrets, infinite loops, memory, processes, output floods, disk floods, tricks against the test harness	Every one **contained**, **never reported as passed**; **server stays up**; **no secret or test file exposed**; **next normal submission still runs correctly**	Hand-write all 15. Add "after each, submit a good program" to the loop
T2	4 programs × 3 challenges: pass / wrong output / error / run forever → **12 labels**	**All 12 exactly right**	Build the 12 corpus cases now; assert labels
T3	Programs that **print or raise their inputs**, request hints, **inspect browser traffic**	No hidden input, expected, output or exception text ever reaches the learner	Assert on the JSON: `hidden[]` may only contain `test` + `outcome`. Diff every response body for known hidden strings
T4	3 failing submissions → request **every hint**, even after asking for the answer → **merge all code from the hints into the learner's code**	**Merged code still fails ≥1 hidden test**	Auto-test: hints must contain no code-like tokens. Keep a `hints.must_not_compile` unit test
T5	Scripted learner submits **~11 programs** over the 4 judge challenges	**Both planted recurring mistakes named**, one-off slips **not**, and the **next challenge targets a recurring mistake with a reason**	Script the 11 submissions as a fixture; assert on `GET /api/learner`
T6	**2 judges rate 5 failures, blind to team, 0–2 each** on **correct**, **specific**, and **not giving the answer away**	Points scale with the average rating	Have 4 people read your explanations cold and score them 0–2. Anything under 1.5 → rewrite

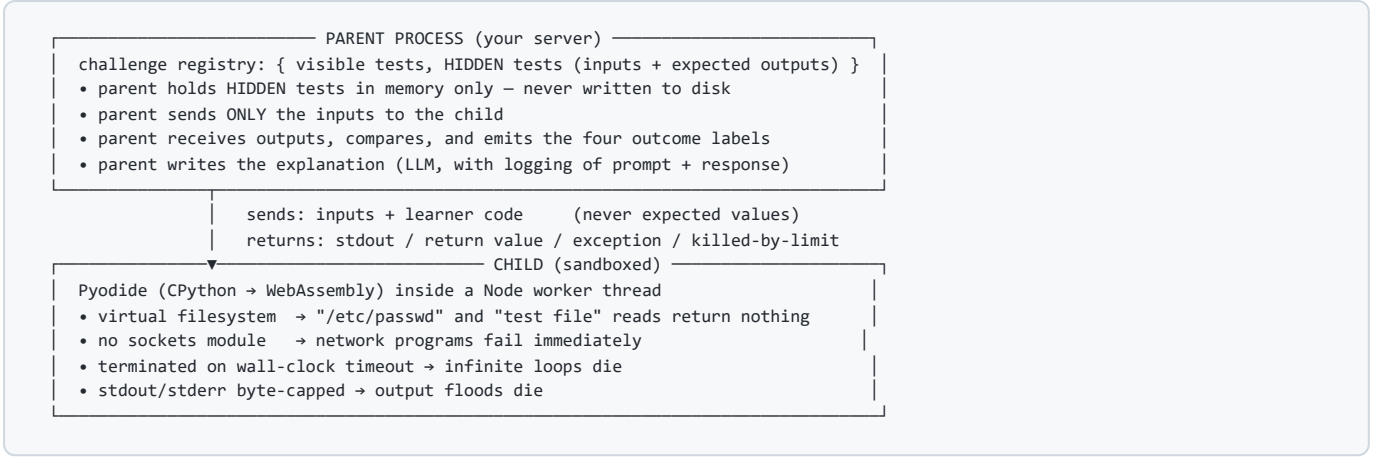
Run the whole harness at 21:00 Day 1, 06:00 Day 2, and 12:00 Day 2. Three runs, and you know your score before they do.

---

7. Architecture — the two decisions that win it

7.1 Decision 1: hidden tests live only in the parent's memory

This is the single highest-value design choice in the project.



Why this wins three tests at once:

- **T1 (red team):** file/secret reads fail *by construction* — there is no host filesystem in WebAssembly. There is no test file to leak, because it never existed on disk.
- **T3 (secrecy):** the child never receives expected values, so it cannot echo them. Your browser never receives them either, so browser inspection finds nothing.
- **T1 "tricks against the test harness":** the harness source in the child has no access to the parent's memory.

Defence in depth (this is the "stronger isolation scores higher" bonus): run the Pyodide worker as a **spawned child process** with OS limits applied — `RLIMIT_CPU`, `RLIMIT_AS`, `RLIMIT_FSIZE`, `RLIMIT_NPROC` — and kill the **process group** on wall-clock timeout. Two independent layers: WASM sandbox *and* OS limits.

Verified today: `pyodide@314.0.7` is published on npm and installs with your npm 11.19. Node v26.10 is on your machine. **Test this dependency first — it is your only real technical risk.**

Fallback (if Pyodide's download size or speed fails on venue Wi-Fi): a `python3 -I -S` subprocess with rlimits in a clean temp dir, with `socket` blocked and `open()` restricted to that temp dir. Weaker, but still passes a *constrained* runner bar — which the statement explicitly permits.

7.2 Decision 2: outcome labelling as a pure function

Exactly one label per test, deterministic, no guessing:

```
kill reason          → label           → extra field
normal exit, match   → passed          →
normal exit, differs → wrong_output    →
raised / bad exit    → runtime_error   → exception class (learner's own, hidden-test side is empty)
RLIMIT_AS hit        → runtime_error   → "memory limit"
RLIMIT_FSIZE hit     → runtime_error   → "output limit" / "disk limit"
RLIMIT_CPU hit       → runtime_error   → "cpu limit"
wall-clock exceeded  → timeout         →
```

Never let two conditions race: check resource kill **before** wall-clock, and wall-clock **before** reading output.

7.3 Decision 3: label corpus before features

T2 is worth 12 labels and it is **fully knowable**. Build the 12 programs (3 challenges × {pass, wrong output, error, run forever}) as fixtures in the first two hours, and assert the classifier hits 12/12. Then add the 15 hostile programs from T1. Your scoreboard now exists at 18:00 on Day 1 instead of 06:00 on Day 2.

7.4 The hint ladder — the part with human raters

The statement's own model ladder (note: **no code anywhere**):

1. *"Your three examples never ask for more marks than the list holds. What should* `last_k_total` *return when k is bigger than the list?"*
2. *"Work out* `last_k_total([2, 5], 3)` *by hand. Which index does your loop start at, and what does* `marks[-1]` *mean in Python?"*
3. *"In Python a negative index counts back from the end, so some marks get added twice. Your start index must never go below 0."*
4. *"Python has a built-in that returns the larger of two numbers. Use it to work out the start index once, before the loop."*

Rules to enforce in code, not in prompt-writing:

- Store hints as **prose only.** Reject any hint containing code fences, `def`, `=`, `()`, or an import. This makes T4 mechanical instead of hopeful.
- Hint 4 may *name a builtin* (`max`) but never show how to use it.
- If the learner writes *"give me the answer"* → still escalate one rung, never collapse the ladder.
- **Log every prompt and response** (the rules require it) — ship it as a browsable **"Hint Ledger"** page. A compliance requirement turned into a demo feature judges will actually click.

T6 scoring: 2 blind judges × 5 failures × 3 axes × 0–2. Score yourself before they do — have four people read them cold.

---

8. The five detectors — your one-page deliverable

This page is **an explicit deliverable**, so build each detector with its own planted-buggy-program unit test.

Category	Detection rule (deterministic, no LLM)
`unhandled_short_input`	The learner's code raised `IndexError` / `TypeError` **and** the submitted input is shorter than the contract requires (empty or < expected length). Classify from exception type + input length — **never expose the hidden values**
`print_not_return`	The program printed the correct value to stdout **but** returned `None`
`mutates_input`	Snapshot the input before and after the call; if mutated, or the second identical call returns a different result → this tag
`off_by_one`	Numeric result differs from expected by exactly **±1**, and the failing input is **not** at a boundary edge
`boundary_comparison`	Numeric result differs by ±1 **and** the failing input **is** at an edge case: `k == len`, `k > len`, `n == 0`, `len(items) < 3`, or empty list

Add one more if you have time — `wrong_index_direction` is a natural sixth, and *"may add more"* is explicitly permitted.

Recurring logic: roll up tags **across submissions**, keyed by learner. A tag is recurring only if it appears in **≥2 different challenge IDs** — count challenges, not submissions. A learner who fails the same challenge five times has *one* challenge, not two.

Deliverable format: one page, table of `category → detection rule → example bug → unit test name`. Keep it printable.

---

9. Mastery model — simple beats clever

Judges ask "how does it work?", not "prove it converges". Use a Beta-ish prior per concept and explain it in one breath:

```
prior  α=1, β=1  (mean 0.5)
pass, no hints        α += 1.0
pass, after hints     α += 0.35   ← penalty is the point
fail                  β += 0.6
given a hint          β += 0.15   ← each hint costs
mastery = α / (α + β)
```

Mastery must be **per concept** (concept tags on every challenge), visible in `GET /api/learner`, and explainable in 10 seconds. That is all T-scoring needs.

---

10. The challenge bank (write it in hour 1–2)

Difficulty	Concept	Challenge	Judge one?
Easy	lists / indexing	`count_passes`	✅
Easy	loops / range	`first_n_squares`	✅
Medium	lists / slicing	`middle_three`	✅
Medium	boundary / accumulation	`last_k_total`	✅
Easy	strings	`is_palindrome`	
Easy	conditionals	`grade_letter(marks)`	
Medium	lists	`second_largest(items)`	
Medium	loops	`sum_of_digits(n)`	
Medium	strings	`count_vowels(text)`	
Hard	boundary	`rotate_left(items, k)`	
Hard	lists / edge cases	`merge_sorted(a, b)`	
Hard	conditionals	`is_leap(year)`	

→ **12 challenges**, 4 difficulty levels, 6 concepts, 12 × (3 visible + ≥5 hidden) tests. Each hidden set must include an edge case (empty, zero, negative, single element, larger-than-list).

---

11. Your 13.5-hour plan

Time	Deliverable	Gate
14:20–15:30	**Environment only — no project code** (§14)	
15:30	Clock starts. Repo + first commit. `POST /api/submit` returns the JSON shape with **hardcoded** challenge data	
15:30–17:00	**Pyodide child runner**: execute code, catch exceptions, wall-clock timeout, byte cap	
17:00–18:00	**Classifier: 12/12 on the label corpus** + 5 hostile programs	
18:15	Eval 1 prep: one-page architecture diagram (parent holds hidden tests — draw it)	
18:30	**Eval 1** — present architecture + isolation story. Ask the 3 questions in `README.md` §3.3	
18:30–20:00	Remaining 10 hostile programs → **T1 green**	
20:00–21:00	12 challenges authored with full test sets	
21:00	**Run the full harness (T1–T5).** Record the score. Everything red is tonight's backlog	
21:00–23:00	Explanation pipeline (LLM + logging) + hint ladder storage (prose-only guard)	
23:00	**GATE LOCKS. Feature freeze.** Deploy, commit, screenshot, push	
23:00–00:30	`POST /api/hint` + `GET /api/learner` + recurring rollup across challenges	
00:30–06:00	**Sleep in 2 shifts.** Non-negotiable — you have a viva at 15:30	
06:00–07:30	Mastery model + learner UI + the **Hint Ledger** page	
07:30	**Run the full harness again.**	
08:00	**Round 1** — core functionality and progress	
08:00–10:00	Fix only what judges flagged	
10:00	**SCOPE LOCK. No new features**	
10:00–12:00	UI polish, one-command startup, README, taxonomy one-pager	
12:00	**Final harness run.** Fix reds only	

Time	Deliverable	Gate
12:00–13:00	Rehearse 5-min demo twice back to back	
14:00	Code freeze	
14:00–15:00	Eat, water, all four present and awake	
15:30	**Round 2 — 5-min demo, then viva**	

The three harness runs (21:00, 07:30, 12:00) are the load-bearing moments of your event. Protect them.

---

12. Ownership split — scales to your team size

Person	Owns	Never touches
A — Runner	Pyodide child, rlimits, timeout, byte caps, **T1's 15 hostile programs**	Prompts
B — Logic	Classifier, detectors, recurrence rollup, mastery, **T2's 12 cases, T5's 11-script fixture**	UI
C — Product	Learner UI, explanation display, mastery visuals, **Hint Ledger page**, responsive/layout	Runner internals
D — Spec/QA	Challenge bank + hidden tests, prompt design + logging, **the harness script itself**, README, one-pager, deck	Core logic

Your team is proper-sized — scale it, don't trim it: add a 5th owner as **harness operator** (runs §6 at 21:00 / 07:30 / 12:00, nobody else touches it) and a 6th as **demo/stage manager** (owns the §13.2 speech timing, keyboard handoffs, the live red-team paste). More people = more parallel verification, which is the scarce resource (§1).

---

13. The demo, the speech, and the business

13.1 The 5-minute demo (the emotional arc)

Open on **their own story** — the judges wrote it, so they will feel it:

Time	Beat
0:00–0:25	Lead with the speech in §13.2 — then: *"Rohan's function passed all three examples and failed hidden test 5."* Submit it live.
0:25–1:30	Submit the buggy `last_k_total`. Show **3 visible passing / hidden failing**, then the explanation — plain language, names the *kind* of input, **no values**.
1:30–2:30	The **hint ladder**, four rungs live. Then ask *"give me the answer"* → still one rung at a time. Then show the **Hint Ledger** proving zero code in every hint.
2:30–3:30	**The wow:** open `GET /api/learner` — *"you keep making `unhandled_short_input` across three different challenges, so here's the next problem and why."* Mastery per concept updating.
3:30–4:15	**Red team, live:** paste a hostile program that tries to read `/etc/passwd` and the test file → **blocked**, label correct, server still up, and the next good submission still runs.
4:15–5:00	One page: taxonomy + how each category is detected. The 15/12/11 numbers. Close: *"We never gave the answer. We gave the place to look."*

Viva prep (they will probe): why Pyodide, how recurrence is counted, why hints contain no code, how you prevent hidden-test leakage, what mastery optimises for, what you'd do next (per-learner spaced repetition).

13.2 The speech — say this before the first click (90 seconds)

> Two hours from now you will look at a student's code and know immediately what is wrong with it. You have done that a thousand times.
>
> That student has no idea. And he will not ask you, because asking costs him the one thing he thinks he cannot afford in front of his peers — the appearance of not knowing.
>
> So tonight he will paste his question into a chatbot. It will hand him the answer in four seconds. He will submit it, he will move on, and tomorrow he will not be able to do the thing he just handed in.
>
> The judges wrote that story into this problem statement. We think every faculty member in this room has met Rohan — the boy who learned to copy instead of learn, because nobody taught him the thing that comes *before* the answer.
>
> We did not build another chatbot that gives answers faster. We built the one that refuses —
>
> *[turn to the screen]* — and this is what refusing looks like.

How to deliver it:

- **Slow.** This is not a pitch speed. Pause after *"the appearance of not knowing."*
- **Eye contact for the first 45 seconds** — no slides, no screen. The judges are faculty; they have lived this.
- **One person speaks.** The other hands are already on keyboards, ready to run the demo the instant you say *"this is what refusing looks like."*
- **The turn is the cue.** Your keyboard operator starts on the word *"looks."*
- Never say "AI-powered" or "leverage." Say *student*, *hint*, *mastery*, *refuses*.

The closer (last 20 seconds of the demo):

> Rohan never needed the answer. He needed someone to show him where to look — and then to get out of the way.
>
> Every hint we have ever written is in that ledger, and not one of them contains a line of code. The endpoints are live. Ask us anything.

13.3 The business — who pays, and why it compounds

(Not on the published rubric — this is your viva answer and your "potential" story. Do not build it instead of §6. Everything below is a slide, not a sprint.)

The product in one sentence: a debugging tutor that provably never hands over the answer — sold to the institutions whose entire assessment model just broke.

Why now (the real-world hook): ChatGPT broke the take-home assignment. Every college, bootcamp and training institute is asking *"how do we certify that a student actually learned this?"* — and there is no product answer today. **Debug Dojo is that answer, because it isn't a detector, it's a teacher.**

Tier	Buyer	What they get	Why they pay
Free	learners	challenges + hint ladder + mastery map	growth loop — a shareable mastery map posts itself
Coach *(₹/seat/mo)*	tutors, mentors, bootcamp instructors	assign challenges, see **recurring-mistake reports per student**	the thing they currently do in a spreadsheet at 11pm
Institution *(annual)*	colleges, training institutes, placement cells	cohort dashboards, **no-answer-guarantee audit trail**, API	the "did they actually learn it?" question, answered and provable
White-label	edtech platforms, publisher courseware	the engine inside their content	they already have the students; they lack the ethics layer

The four features that "attract support" — all already in your build:

1. **The Hint Ledger** → repackaged as a **trust artifact**. *"Ask us to show you every hint we've ever written."* It's a compliance requirement that becomes a marketing weapon.
2. **Mastery map per concept** → the retention feature. Students come back to move a bar; the bar is the product.
3. **Recurring-mistake report** → the sales feature. A tutor sees *"class average: boundary_comparison, 61%"* — that is a classroom purchase order.
4. **Shareable, pseudonymous learner profile** → the acquisition feature. No real student data (rules-compliant), but a link worth posting.

The moat: competitors can copy the UI, not the **corpus of logged hints and outcomes**. Every submission makes the mistake-taxonomy detection sharper — a data flywheel that a new entrant cannot buy.

If a judge asks *"is this a business or a hackathon project?"*:

> *"Both, and the same sentence covers it: institutions will pay for proof that their students learned instead of pasted. We shipped the proof — the ledger — as a product feature on day one."*

---

14. First 75 minutes (14:35 → 15:30) — environment only

⚠️ **No project code before 15:30.** The rule is *"work must be created during the hackathon."* Dependencies, data and environment are preparation — writing your app is not.

Claude Code Pro note: your AI writes the code *after* kickoff — the pre-kickoff window is for the **environment it will run in.** If Pyodide isn't proven working before 15:30, every generated line is built on an unverified foundation.

- [ ] `npm i pyodide` and **prove it runs a 5-line Python snippet inside a worker, times out on an infinite loop, and cannot open a file.** This is your only real technical risk — resolve it before you sit down.
- [ ] Test the fallback path once: `python3 -I -S -c "1/0"` with a 1s timeout from Node.
- [ ] Get **2 LLM API keys working** (one backup). Required for explanations/hints. Confirm they work.
- [ ] `pip install gitleaks` equivalent — install **gitleaks**, `git init`, verify `.env*` is ignored.
- [ ] Download the datasets **allowed for content**: MBPP (CC BY 4.0), HumanEval (MIT), Exercism Python (MIT). You may adapt openly licensed content **with attribution** — hidden tests must be your own.
- [ ] Pre-clone any template you'll start from (a bare Next.js/Express skeleton as *tooling*).
- [ ] Pack: laptop + charger + adapter, **power bank**, **extension board**, valid college ID, notebook, water, headphones. **No shorts.**
- [ ] Alarms: 18:25 · 21:00 · 22:55 · 07:25 · 11:55 · 13:55
- [ ] Read the 6 judge tests (§6) one final time out loud as a team.

---

15. Risks & mitigations

Risk	Mitigation
Pyodide fails on venue Wi-Fi (cold download)	Install and cache **before 15:30** (§14). Ship the subprocess fallback (§7.1)
Pyodide too slow for the scripted learner	Cache the runtime; pre-warm the worker; the 11-script fixture has no latency requirement
Hidden values leak into a response	Automated test: assert no hidden string ever appears in any response body (T3)
LLM writes code into a hint	Prose-only validator rejects code-like tokens at **storage time**, not at generation time
Recurrence false-positives (one-off slips flagged)	Count **distinct challenge IDs**, not submissions. Add the T5 fixture
Judges can't reach your endpoints	Deploy by 20:00, **and** bring a laptop running the same command
Night shift kills your viva	Sleep 2 shifts. Nobody demos at 15:30 on 2 hours' sleep
Scope creep after 10:00 Day 2	Scope lock. New ideas go on the "what we'd do next" slide, never the build
Plagiarism doubt	Commit every 30 min from 15:30; repo public; README credits every adapted source

---

16. The other four DI statements — and why not

Statement	Why you're not taking it
DI-06 Pay-per-call	Your friend has it and every team's GPT recommends it → the crowd. **Dead as a pick.**
DI-04 Chimney Census	The "crazy" option: a public leaderboard you can watch overnight, 2 submissions/hour, and an Inspector's Brief naming the 20 kilns she should visit first. **But** it needs a T4 GPU and a 4 GB download before you can start. **Take it only if, by 15:00, you have a working GPU runtime and ≥30% of the dataset on disk.** If not, it's a 0
DI-02 NeuroScope 3D	Must land within **0.05 Dice** of the organisers' baseline on hidden cases, with correct affine maths, anisotropy handling and an 8-case failure pack. Requires MONAI-level expertise. Not a 13-hour project
DI-03 LiveSub	Must beat the organisers' **WER/CER baseline** in 4 Indian languages at median ≤3 s. Their baseline is Whisper large-v3-turbo + IndicTrans2. Beating it in 13 hours while also handling Manifest V3 tab capture — no

On the Open track: 6 statements, **no test pack**, subjective judging, and the TechSummitTrainers terms assign your code to them permanently. **On Agentic AI:** only 2 statements, so both crowds are dense.

Neither dominates.

---

17. Commitment card

```
TRACK      : Decentralized Intelligence
STATEMENT  : DI-01 — Debug Dojo
LANGUAGE   : Python (one language, supported well)
RUNTIME    : Pyodide in a Node worker + OS rlimits + process-group kill
             fallback: python3 -I -S subprocess, constrained
CORE IDEA  : hidden tests live ONLY in parent memory — never on disk, never in the browser
SCOREBOARD : T1 15 hostile | T2 12 labels | T3 secrecy | T4 no-code hints
             T5 11-script learner | T6 two blind raters
RUNS AT    : 21:00 Day 1 · 07:30 Day 2 · 12:00 Day 2
FREEZE     : 23:00 tonight (features) · 14:00 tomorrow (code)
SLEEP      : 2 shifts after 23:00 — you have a viva at 15:30
DELIVER    : repo + one command · live URL AND laptop · taxonomy one-pager
             5-min demo + viva · §13.2 speech (memorised) · §13.3 business slide
```

> *"Rohan never needed the answer. He needed someone to point him to the right place to look."*

That sentence is from the judges' own problem statement. Finish the story they started — give them the ending, and let them watch it work.